ARYAN PARAB

San Jose, CA | 310-869-1411 | amparab@usc.edu | linkedin.com/in/aryan-parab-0b44991b2

EDUCATION

University of Southern California Los Angeles, CA
M.S. in Computer Science | GPA: 3.85/4.0 January 2025 - December 2026
Coursework: Coursework: Algorithms, Operating Systems, Database Architecture, Web Technologies, Information Retrieval, Robotics
Thadomal Shahani Engineering College Mumbai, India
Bachelor of Engineering, Computer Engineering | GPA: 9.1/10 August 2019 - May 2023

TECHNICAL SKILLS

Languages: Python, Java, C, SQL
Backend: FastAPI, Flask, Node.js, REST APIs, Async Programming
Databases: PostgreSQL, MongoDB, SQLite, DuckDB, LevelDB, LSM Trees
Systems: Linux/Unix, Processes, Threads, Synchronization, File Systems, System Calls, Concurrency
Infrastructure: Docker, AWS EC2/S3, Git, CI/CD, Cron, Playwright

EXPERIENCE

Cadence Design Systems San Jose, CA
Generative AI Software Engineering Intern May 2026 – Present
-Architected a scalable test automation platform spanning 1,000+ QA test cases, converting specifications into executable workflows with planning, simulation, execution, debugging, and recovery.
-Designed a dependency-aware workflow engine that decomposes test cases into reusable components and shared execution dependencies, reducing duplicated setup logic across large automation suites.
-Integrated Playwright with MCP-based Katalon tooling to execute browser workflows, capture runtime failures, generate targeted fixes, and retry failed steps automatically.
-Built a daily Cloudflare traffic-analysis and security recommendation pipeline that ingests 24-hour traffic, filters suspicious requests using WAF/bot scores, and clusters traffic using JA4 fingerprints and IP behavior.
-Developed backend services that correlate detected threats with existing Cloudflare rules, identify rule coverage gaps and failure conditions, and generate actionable recommendations for new or modified protections.
-Automated the end-to-end pipeline using scheduled Cron jobs, including traffic ingestion, preprocessing, analysis, rule evaluation, report generation, and daily delivery to the security team.

PROJECTS

LevelDB BitWeaving & ZoneMap Optimization
C++, LevelDB, LSM Trees, SIMD, Storage Systems April 2026
-Modified LevelDB's LSM-tree storage engine to add per-block ZoneMap metadata and compact bitmask summaries, enabling selective block skipping before disk I/O during range queries.
-Integrated metadata generation into the write/compaction path and lookup logic into the read path, allowing blocks to be rejected before decompression and scanning.
-Used SIMD-assisted metadata scanning and bitmask filtering to accelerate predicate evaluation across blocks while adding only ~0.3% storage overhead.
-Benchmarked the system from 1M to 500M records across synthetic and time-series workloads, validating scalability under large datasets and concurrent reads/writes.
-Achieved up to 31.2× query speedup with 92.3% of blocks skipped on selective 10M-record IoT workloads, and sustained roughly 25–28× speedups with 96.42% I/O reduction under concurrent read/write execution.

Weenix Operating System Kernel — Systems Programming
C, Linux, GDB, Concurrency December 2025
-Implemented Unix-like kernel functionality across process/thread management, scheduling, synchronization, VFS operations, and system calls within a 10,000+ line C codebase.
-Implemented process lifecycle functionality including fork(), exec(), waitpid(), scheduling, and thread synchronization.
-Built and debugged filesystem functionality including file operations, directory management, path resolution, and VFS abstractions.
-Diagnosed race conditions, process lifecycle bugs, deadlocks, and filesystem failures using GDB, kernel logs
-Validated concurrent execution across workloads containing 100+ processes.

Idol Coach — Full-Stack Audio Processing Platform
React, Next.js, FastAPI, PostgreSQL, MongoDB, AWS, Docker May 2025
-Built a full-stack web application for recording, processing, and analyzing user audio through a React frontend and FastAPI backend.
-Designed REST APIs for audio uploads, asynchronous processing, session management, and retrieval of processed results.
-Integrated PostgreSQL and MongoDB for user/session data and AWS S3 for persistent audio storage.
-Built an audio-processing pipeline using Librosa and lyric alignment services to compare user recordings against reference vocals.
-Optimized backend processing and storage workflows, reducing inference and processing latency by approximately 30%.